Ateneo de Zamboanga University
College of Science, Information Technology and Engineering
Computer Science Department
FINAL EXAMINATION

Arithmetic Expression Validator

AUTOMATA: Automata Theory

Epino, Rhett Ushley E.

Gler, John Matthew S.J

Gonzalez, Jessie James C.

Recto, Sean Benedict D.

[Instructor Name]

[Date Submitted: MM/DD/YYYY]

Abstract

This project addresses the scenario of a calculator application that must validate an arithmetic expression before evaluating it. Valid expressions are built from the digits 0–9, the operators +, −, *, and /, and parentheses. It must also follow the rules that parentheses are balanced and properly nested, no operator appears at the start or end, no two operators are consecutive (except a unary minus), unary minus is allowed only at the start or after an opening parenthesis or another operator, and a digit sequence has no leading zeros unless it is exactly 0. After analyzing the language, we classified it as context-free but not regular, because checking arbitrarily deep nested parentheses requires an unbounded stack; a pumping-lemma argument confirms that no finite automaton can recognize it. We therefore wrote a context-free grammar with seven nonterminals and a pushdown automaton (PDA) with four states and the stack alphabet {Z, (}. The PDA was implemented in C++ as a state-transition function over three working states plus a trap state, together with a std::stack of open parentheses. The simulator was tested against 53 representative cases covering the examples in the problem statement, nesting, operator placement, unary minus, leading zeros, and malformed input.

Abstract....2
I. Introduction....4
Problem Statement....4
Objectives....4
Scope and Assumptions....4
II. Formal Language....4
Alphabet....4
Language Definition....4
Examples....4
Edge Cases....4
III. Classification....4
IV. Grammar....5
Definition....5
Derivations....5
V. Automaton....5
Definition....5
State Diagram....5
Transition Table....5
VI. Implementation....6
Tools and Environment....6
Design....6
Testings....6
Limitations....6
Appendix A: Division of Work....7
Appendix B: Source Code....7

I. Introduction

Problem Statement

A calculator application needs to validate arithmetic expressions before evaluating them. An expression is a string over the digits 0 through 9, the operators +, -, *, /, and the parentheses (and). The system must accept exactly the strings that satisfy all of the following rules, and reject every other string:

- Parentheses must be balanced and properly nested.
- Operators cannot appear at the start or end of an expression.
- Two operators cannot appear consecutively.
- Unary minus is allowed only at the start of an expression or immediately after an opening parenthesis or another operator.
- A digit sequence cannot have leading zeros unless the sequence is exactly 0.

The given examples are: 3+4, (3+4)*(2-(1+5)), -5+3, and 0+1 (accept); 3++4, +3, 3+, (3+4, 007+1, and 3*(/2) (reject).

Objectives

1. Define the syntax of valid arithmetic expressions using set builder notation.
2. Classify the language in the Chomsky hierarchy along with supporting reasoning.
3. Design a context-free grammar which describes the language.
4. Design a pushdown automaton which recognizes the language.
5. Design a simulator for the pushdown automaton using C++.
6. Test the simulation using representative, boundary, and bad input test cases.

Scope and Assumptions

To keep the problem tractable, the following simplifications are made:

- The input will be ASCII only; anything but the 16 characters of Σ will be rejected.
- The validator checks syntax only. It does not evaluate the expression, apply PEMDAS, or reject mathematically invalid expressions such as 1/0.
- Unary plus is not allowed since it is forbidden in the problem statement. Unary minus is the only allowed unary operator.
- Since the problem allows unary minus after an operator, the same is allowed for multiple unary minuses like --5 and 3 --- 4.
- All numbers will be positive integers, signs are introduced by unary minus.

Explicitly out of scope:

- Decimal points, exponents, variables, functions and operators.
- Any whitespace inside or around the expression (spaces are invalid characters).
- Implicit multiplication, like 2(3), (1)(2).
- Evaluating the expression or reporting a numeric result.

II. Formal Language

Alphabet

The alphabet is the set of digits, operators, and parentheses:

$$\Sigma = \{ 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, +, -, *, /, (,) \}, \text{ so } |\Sigma| = 16.$$

For convenience we write D = { 0, …, 9 } for the digits and O = { +, −, *, / } for the operators.

Language Definition

Let $L \subseteq \Sigma^*$ be the language of all strings over Σ which form syntactically correct arithmetic expressions. First, we will need the sets of numbers not starting with zero and binary operations, respectively:

$$N = \{ 0 \} \cup \{ d_1 d_2 \ldots d_k \mid k \geq 1,\ d_1 \in \{1,\ldots,9\},\ d_i \in \{0,\ldots,9\} \text{ for } i \geq 2 \}$$

$$O = \{ +, -, *, / \}$$

Primaries include both numbers and parenthesized expressions, and the language L satisfies the following set equations:

$$P = N \cup \{ (v) \mid v \in L \}$$

$$L = \{ u_1 p_1 o_1 u_2 p_2 o_2 \ldots o_{k-1} u_k p_k \mid k \geq 1,\ u_i \in \{ - \}^*,\ p_i \in P,\ o_i \in O \}$$

Each u_i stands for a possibly non-empty sequence of unary minus signs before the corresponding operand. As L is defined as the smallest solution of these equations, parentheses are nested finitely many times.

In natural language: The language L contains all strings over Σ which describe arithmetic expressions calculable by a calculator machine: a sequence of operands interspersed by binary operators, where each operand is either a number not starting with zero or a parenthesized expression. An operand can be preceded by a non-empty sequence of unary minus

signs, but such operand must be at the beginning of the expression or after a binary operator or an opening parenthesis.

Using a notation of a kind of regular expressions, with U = -* standing for a possibly non-empty sequence of unary minus signs and P = N | (L) denoting a primary:

$$L = U\ P\ (\ o\ U\ P\ )^* \qquad \text{with}\ \ o = + \mid - \mid * \mid /$$

Examples

The following are 5 strings in L.

String	Justification
3+4	A number, an operator, and then another number. Parentheses not needed, and no leading zeros.
(3+4)*(2-(1+5))	Parentheses are balanced and nested appropriately. operators are in the correct positions.
-5+3	The unary minus at the start of the expression is allowed. It then continues by alternating operand and operator.
0+1	Single zero counts as a number, hence the leading zero rule is not broken.
5*-3	The '-' after '*' is a unary minus. '-' can come after an operator

The following are 5 strings not in L.

String	Justification
3++4	There are two successive operators. The second '+' is not legal since only a successive unary minus is legal.
+3	Operator at the beginning of the expression, and unary plus is not allowed.
3+	Operator at the end of the expression.
(3+4	The parentheses are not balanced since the '(' is never closed.
007+1	007 is not allowed as it starts with a leading zero.

Edge Cases

Case	**Input**	**Result**	**Reason**
Empty string	""	Reject	The expression must contain at least one operand; the machine is still in q0 when the end marker arrives.
Single symbol, digit	"0"	Accept	A single 0 is a complete expression.

Single symbol, digit	"5"	Accept	A single number is a complete expression.
Single symbol, operator	"+"	Reject	The operator appears in the beginning and end of the expression
Single symbol, unary minus	"-"	Reject	Unary minus cannot be treated as an operand. An operand is expected at the end of input.
Single symbol, parenthesis	"("	Reject	The parentheses are not balanced.
Shortest valid input	"0"	Accept	One character, one operand.
Longest valid input	—	—	The language is unlimited in length: the string "1+1+1+…+1" will always be correct for any given length.
Maximum nesting depth	Unbounded ((…((1))…))	Accept	"((…((1))…))" is correct for any nesting depth since the stack gets one marker for each level.
Too many closing parentheses	"3+4)"	Reject	The ')' has no matching '('. The machine will enter a dead state.
Starting with closing parenthesis	")("	Reject	The stack is empty when reading ')', parentheses underflow.
Empty group	"()"	Reject	A parenthesized expression must include at least one operand.
Sequence of digits 0	"0"	Accept	The sequence is exactly 0, which is the one exception to the leading-zero rule.
Sequence of digits 00	"00"	Reject	The sequence 00 has a leading zero.
Number only	"42"	Accept	A single number is a complete expression.
Two operands in a row	"2(3)"	Reject	Implicit multiplication is not allowed.
Character outside the alphabet	"3 4"	Reject	A space is not in Σ.
Leading zero after minus	-007	Reject	Leading zero after minus.
Consecutive unary minus	–5	Accept	Allowed by our interpretation as a double negative number could be seen as positive, as well as triple negative going back to a negative number.
Division by zero	1 / 0	Accept	Valid syntactically

III. Classification (pick one dito)

- **L is context-free, but not regular.**

L is context-free because the only non-local rule in it describes properly nested parentheses, and nesting is exactly what is described by the context-free productions: the production F → (E) puts an expression in parentheses regardless of its nesting level. Other rules in L (operator placement, unary minus, leading zeros) limit themselves to adjacency, and therefore are compatible with standard productions. A pushdown automaton encodes the same idea in the way that it pushes a symbol for every "(" encountered and pops one for every ")".

L is not regular because it cannot check the rule of correct balance of parentheses. The other rules (operators order, unary minus, leading zeroes) only handle the previous symbol or a symbol just before the immediately previous one, which can easily be managed by a finite automaton. But checking any number of correctly matched "(" and ")" needs counting, which is beyond the power of a finite automaton as it has only a finite number of states.

Pumping-lemma proof. Suppose L were regular with pumping length p. Consider $w = (^p 1)^p$, which is in L and has length $2p + 1 \geq p$. By the pumping lemma, $w = xyz$ with $|xy| \leq p$ and $|y| \geq 1$, so y consists only of (characters, say $y = (^k$ with $k \geq 1$. Then $xy^2z = (^{p+k} 1)^p$ has more (than), so it is unbalanced and not in L. This contradicts the pumping lemma, so L is not regular.

The stack is what the PDA adds: it remembers how many parentheses are currently open. The fixed local rules are handled by the finite control of the PDA.

- **L is context-free but not regular.**

L is context-free because its only non-local rule is nested parentheses, which a production like F → (E) captures at any depth. The other rules only constrain adjacent symbols, so ordinary productions handle them. A PDA does the same by pushing on each (and popping on each).

L is not regular because matching an unbounded number of (with) requires counting, and a finite automaton has only finitely many states. The other rules only look at neighboring symbols, which a finite automaton can handle.

Pumping-lemma proof. Suppose L were regular with pumping length p, and take $w = (^p1)^p \in$ L. Any split $w = xyz$ with $|xy| \leq p$ and $|y| \geq 1$ puts y inside the leading (characters, so xy^2z has more (than) and is not in L. This contradicts the pumping lemma, so L is not regular.

IV. Grammar

L is generated by the context-free grammar G = (V, Σ, P, S), where V = { E, F, O, N, B, R, D } is the set of nonterminals, Σ = { 0, 1, …, 9, +, -, *, /, (,) } is the set of terminals, S = E is the start symbol, and P consists of the following productions:

Production	Nonterminal	Meaning
E → F | F O E	E, expression	An expression is one operand, or an operand, a binary operator, and another expression. It therefore always starts and ends with an operand.
F → N | - F | (E)	F, factor (operand)	An operand is a number, a number with a unary minus in front of it (any number of minuses), or a parenthesized expression.
O → + | - | * | /	O, binary operator	The four binary operators.
N → 0 | B R	N, number	A number is exactly 0, or a nonzero digit followed by any digits.
B → 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9	B, nonzero digit	A single digit from 1 to 9. B is used only as the first digit of a multi-digit number ((N → B R) which also excludes 0.
R → ε | D R	R, remaining digits	Zero or more digits.
D → 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9	D, any digit	An integer from 0 to 9. D is used only after the first digit (inside R). A 0 is allowed, as the 0 of 10 or 100 .

G only defines the syntax of expressions that are valid. And operator precedence or associativity is outside of its scope.

Properties of G

Theorem 3. L(G) = L.

For a nonterminal X write $\mathcal{L}(X)$ for the set of terminal strings derivable from X. Reading the productions bottom-up: $\mathcal{L}(D) = \{0,\ldots,9\}$, $\mathcal{L}(R) = \mathcal{L}(D)^*$, $\mathcal{L}(B) = \{1,\ldots,9\}$, and $\mathcal{L}(N) = \{0\} \cup \mathcal{L}(B)\mathcal{L}(D)^*$, which is exactly the set N of the language definition in Section II. Production F → - F can be applied any number of times before F → N or F → (E), so $\mathcal{L}(F) = \{-\}^* \cdot (\mathcal{L}(N) \cup \{ (w) \mid w \in \mathcal{L}(E) \})$, which is U·P with U = {-}* and P as in Section II. Unrolling E → F O E k − 1 times and finishing with E → F gives $\mathcal{L}(E) = \mathcal{L}(F)(\mathcal{L}(O)\mathcal{L}(F))^*$. Hence $\mathcal{L}(E)$ satisfies

$$\mathcal{L}(E) = U\ P\ (o\ U\ P)^*, \qquad P = N \cup \{ (w) \mid w \in \mathcal{L}(E) \},$$

which are exactly the defining equations of L in Section II. Both L and $\mathcal{L}(E)$ are the smallest solution of this system, because every derivation is finite, so $\mathcal{L}(E) = L$. ∎

Theorem 4. G is unambiguous: every string of L has exactly one parse tree, and therefore exactly one leftmost derivation.

Proof sketch. Ambiguity would need two different ways to derive the same string, so look at every point where G offers a choice. For F the first input symbol decides: "-" gives F → - F, "(" gives F → (E), a digit gives F → N. For N the first digit decides: 0 gives N → 0, a nonzero digit gives N → B R. For R, a digit gives R → D R and anything else gives R → ε; digit sequences are therefore always taken maximally, because after an operand the only symbols that can follow in any sentential form are an operator, ")" or the end of the string, never a digit. For E, after the operand has been derived the next symbol decides: an operator gives E → F O E and the end of the string (or ")") gives E → F. Finally, the ")" that closes a given "(" is determined by counting, so the sub-expression inside F → (E) is unique. Every choice is forced, so the parse tree is unique. ∎

Remark. G is unambiguous but not LL(1) as written, because E → F and E → F O E start with the same symbol. This does not matter for the project, since the validator uses the pushdown automaton. If an LL(1) parser were wanted, the standard left-factoring E → F E′, E′ → ε | O E′ would give one.

Derivations

Derivation 1: 0+1 (single zero allowed)	E ⇒ F O E ⇒ N O E ⇒ 0 O E ⇒ 0 + E ⇒ 0 + F ⇒ 0 + N ⇒ 0 + B R ⇒ 0 + 1 R ⇒ 0 + 1
Derivation 2: -5+3 (unary minus at the start)	E ⇒ F O E ⇒ - F O E ⇒ - N O E ⇒ - B R O E ⇒ - 5 R O E ⇒ - 5 O E ⇒ - 5 + E ⇒ - 5 + F ⇒ - 5 + N ⇒ - 5 + B R ⇒ - 5 + 3 R

	⇒ - 5 + 3
(3+4)*2 (nesting through F → (E))	E ⇒ F O E ⇒ (E) O E ⇒ (F O E) O E ⇒ (N O E) O E ⇒ (B R O E) O E ⇒ (3 R O E) O E ⇒ (3 O E) O E ⇒ (3 + E) O E ⇒ (3 + F) O E ⇒ (3 + N) O E ⇒ (3 + B R) O E ⇒ (3 + 4 R) O E ⇒ (3 + 4) O E ⇒ (3 + 4) * E ⇒ (3 + 4) * F ⇒ (3 + 4) * N ⇒ (3 + 4) * B R ⇒ (3 + 4) * 2 R ⇒ (3 + 4) * 2

V. Automaton

Definition

L is accepted by the pushdown automaton:

$$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

where:

- States Q = { q_0, q_1, q_2, q_3 }, with |Q| = 4
  - q_0: expecting an operand (a number, (, or a unary -). This is the start state.
  - q_1: inside a nonzero-leading number such as 12; more digits are allowed.
  - q_2: just finished an operand: a single 0 or a). No digit may follow.
  - q_3: accepting state, entered by an ε-move at the end of the input.
- Input alphabet Σ = { 0–9, +, −, *, /, (,) }
- Stack alphabet Γ = { Z, (}
- Start state q_0
- Initial stack symbol Z_0 = Z, the bottom-of-stack marker
- Accepting states F = { q_3 }
- Transition function δ, shown below. Here X stands for any stack symbol in Γ, d for a digit, and o for an operator in O. Any combination not listed is undefined and causes rejection.

Each rule reads δ(state, input, top of stack) = { (next state, replacement for the top of stack) }. A replacement of (X pushes (, ε pops the top symbol, and X alone leaves the stack unchanged.

$$(1)\ \delta(q_0, 0, X) = \{ (q_2, X) \} \qquad \text{single zero}$$

$$(2)\ \delta(q_0, d, X) = \{ (q_1, X) \} \qquad d \in \{1,\ldots,9\}$$

$$(3)\ \delta(q_0, -, X) = \{ (q_0, X) \} \qquad \text{unary minus}$$

$$(4)\ \delta(q_0, (, X) = \{ (q_0, (X) \} \qquad \text{push (}$$

$$(5)\ \delta(q_1, d, X) = \{ (q_1, X) \} \qquad d \in \{0,\ldots,9\}$$

$$(6)\ \delta(q_1, o, X) = \{ (q_0, X) \} \qquad o \in \{+,-,*,/\}$$

$$(7)\ \delta(q_2, o, X) = \{ (q_0, X) \} \qquad o \in \{+,-,*,/\}$$

$$(8)\ \delta(q_1,), () = \{ (q_2, \varepsilon) \} \qquad \text{pop (}$$

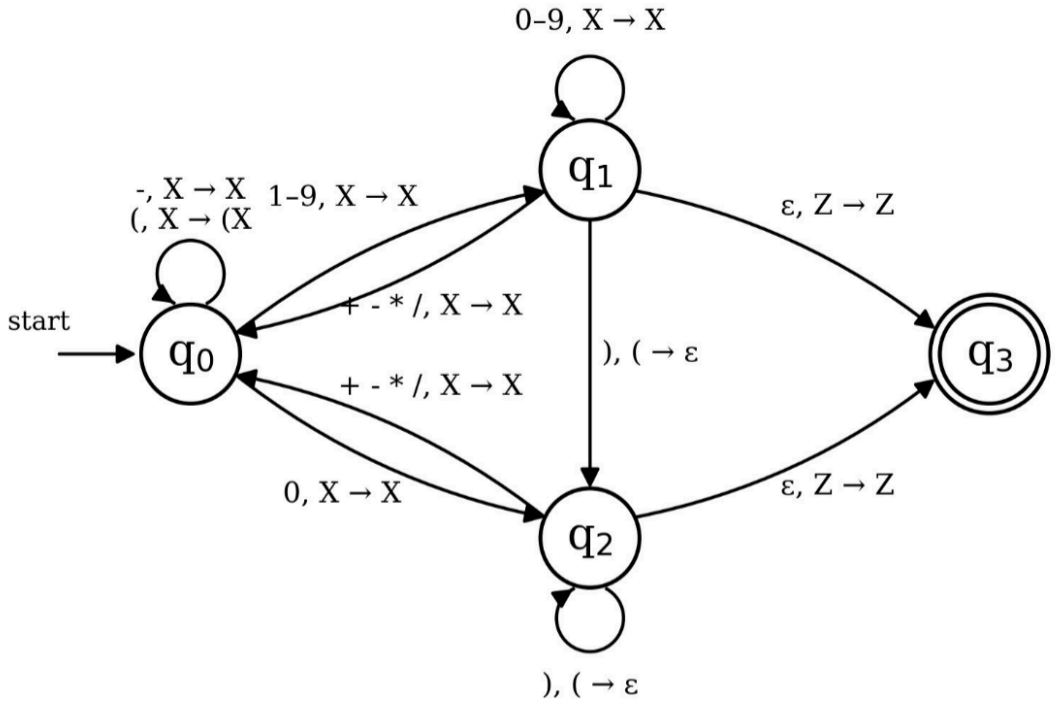
(9) $\delta(q_2,), () = \{ (q_2, \varepsilon) \}$ pop (

(10) $\delta(q_1, \varepsilon, Z) = \{ (q_3, Z) \}$ accept: input done, stack empty

(11) $\delta(q_2, \varepsilon, Z) = \{ (q_3, Z) \}$ accept: input done, stack empty

The PDA accepts by final state: the string must be read completely, and the machine must reach q_3. The transitions (10) and (11) can only be applied if the stack top is Z, which means that all the opening parentheses have been paired with the appropriate closing parentheses. In the given PDA design, there is an extra state qD that acts as a trap for undefined moves.

State Diagram



X ∈ {Z, (} (any stack symbol); label format: input, top of stack → replacement

Transition Table

State	0	1-9	+ * /	0	(	)	ε (top = Z)
→ q_0	q_2	q_1	—	q_0	q_0, push (	—	—
q_1	q_1	q_1	q_0	q_0	—	q_2, pop (	q_3

q_2	—	—	**q_0**	**q_0**	—	q_2, pop (	q_3
q_3 (accept)	—	—	—	—	—	—	—

VI. Implementation

Tools and Environment

Tools and Environment

- Programming language: C++ (C++11 or later)
- Compiler: GNU Compiler Collection (g++)
- Libraries used: only the standard library headers `<iostream>` (console I/O), `<stack>` (the PDA stack), and `<string>` (input handling).

Build and run:

```
C/C++
g++ -std=c++11 -Wall -Wextra -pedantic expr_validator.cpp -o expr_validator

./expr_validator                # interactive menu
./expr_validator "3+4"          # check a single expression
./expr_validator --test         # run the built-in test suite
```

Design

The PDA is represented by three pieces of data and one function:

- **States** are integer constants: `EXPECT_NUMBER` (q_0), `IN_NUMBER` (q_1), `EXPECT_OPERATOR` (q_2), and `DEAD` (qD, the trap state). The accepting state q_3 is not stored; it is represented by the end-of-input check described below.
- **Stack** is a `std::stack<char>` named `parens`. Only `(` is ever pushed. The empty stack stands for the bottom marker Z, so `parens.empty()` is exactly the condition "top of stack is Z".
- **Transitions** are implemented by `nextState(state, c, parens)`, which takes the current state, the input character, and the stack, performs any push or pop, and returns the next state. It replaces a table with if-statements that follow the transition table in Section V.

- **Helpers**: `isDigit`, `isOperator`, and `stateName`, plus `stackToString`, which prints the stack as `Z` followed by one `(` per open parenthesis for the step-by-step trace.

The simulator is `checkExpression(input, showSteps)`:

```
None
state ← EXPECT_NUMBER        (q0)

parens ← empty stack         (only Z)

for each character c in input:

   state ← nextState(state, c, parens)

   if state = DEAD:

      return REJECT          (stop early)

return ACCEPT if state ∈ {IN_NUMBER, EXPECT_OPERATOR}

           and parens is empty

     else REJECT
```

The final check corresponds to the ε-moves (10) and (11): the last symbol must have ended an operand (state q_1 or q_2), and no `(` may remain on the stack. Option 2 of the menu prints every step as (state, input left, stack), which mirrors the instantaneous descriptions of a PDA.

Testings

The program was tested with the following 53 cases. They cover the ten examples of the problem statement, each of the five rules of the specification, the boundary conditions of Section II, and the characters that are not in the alphabet. The Rule column shows which rule of the specification each case checks, so the table also shows that no rule of the specification is left untested.

Rule codes used in the tables:

- R1: parentheses are balanced and properly nested
- R2: no operator at the start or at the end of the expression
- R3: no two operators in a row
- R4: a unary minus only at the start, after (, or after an operator
- R5: no digit sequence with a leading zero unless it is exactly 0
- R6: a well-formed expression: operands and operators alternate
- R7: a boundary condition of the specification

- RX: a character that is not in the alphabet

#	Rule	Input	Expected	Observed	Notes
1	R6	"3+4"	ACCEPT	ACCEPT	a number, an operator, a number
2	R1	"(3+4)*(2-(1+5))"	ACCEPT	ACCEPT	the example: balanced nested parentheses
3	R4	"-5+3"	ACCEPT	ACCEPT	the example: unary minus at the start
4	R5	"0+1"	ACCEPT	ACCEPT	the example: a single 0 is allowed
5	R3	"3++4"	REJECT	REJECT	the example: two operators consecutively
6	R2	"+3"	REJECT	REJECT	the example: operator at the start
7	R2	"3+"	REJECT	REJECT	the example: operator at the end
8	R1	"(3+4"	REJECT	REJECT	the example: unbalanced parentheses
9	R5	"007+1"	REJECT	REJECT	the example: leading zeros
10	R3	"3*(/2)"	REJECT	REJECT	the example: an operator after (with no operand
11	R1	"(3+4)"	ACCEPT	ACCEPT	one balanced group
12	R1	"((1+2)*(3+4))"	ACCEPT	ACCEPT	two levels of nesting
13	R1	"((1))"	ACCEPT	ACCEPT	a group around a group
14	R1	"3+4)"	REJECT	REJECT	one closing parenthesis too many
15	R1	"(()))"	REJECT	REJECT	an empty group and a stray)
16	R1	")("	REJECT	REJECT	a closing parenthesis with no opener
17	R1	"(1+2))"	REJECT	REJECT	the second group is never closed
18	R1	"()"	REJECT	REJECT	an empty group has no operand
19	R2	"*2"	REJECT	REJECT	an operator at the start
20	R2	"/2"	REJECT	REJECT	an operator at the start
21	R2	"3*"	REJECT	REJECT	an operator at the end
22	R2	"3/"	REJECT	REJECT	an operator at the end
23	R2	"-"	REJECT	REJECT	a lone minus is not an operand
24	R3	"3**4"	REJECT	REJECT	two multiplication signs
25	R3	"3//4"	REJECT	REJECT	two division signs
26	R3	"3+-4"	ACCEPT	ACCEPT	a unary minus may follow an operator

27	R3	"3--4"	ACCEPT	ACCEPT	a unary minus may follow an operator
28	R3	"3+*4"	REJECT	REJECT	the second operator is not a minus
29	R3	"(*2)"	REJECT	REJECT	an operator straight after an opener
30	R4	"5*-3"	ACCEPT	ACCEPT	a unary minus after an operator
31	R4	"3*(-2)"	ACCEPT	ACCEPT	a unary minus after an opening parenthesis
32	R4	"-(3+4)"	ACCEPT	ACCEPT	a unary minus on a group at the start
33	R4	"5/-3"	ACCEPT	ACCEPT	a unary minus after a division sign
34	R4	"(-3)"	ACCEPT	ACCEPT	a unary minus inside a group
35	R5	"0"	ACCEPT	ACCEPT	the single digit 0 is allowed
36	R5	"0-5"	ACCEPT	ACCEPT	a zero followed by an operator
37	R5	"01"	REJECT	REJECT	a leading zero
38	R5	"00"	REJECT	REJECT	leading zeros
39	R5	"000"	REJECT	REJECT	leading zeros
40	R5	"10"	ACCEPT	ACCEPT	a zero inside a number is fine
41	R5	"100+20"	ACCEPT	ACCEPT	zeros inside numbers are fine
42	R6	"42"	ACCEPT	ACCEPT	a single number is a complete expression
43	R6	"10/2"	ACCEPT	ACCEPT	a two-digit operand
44	R6	"1234567890"	ACCEPT	ACCEPT	a long number
45	R6	"1+2+3+4+5"	ACCEPT	ACCEPT	a chain of binary operators
46	R6	"1/0"	ACCEPT	ACCEPT	only the syntax is checked, not the value
47	R6	"2(3)"	REJECT	REJECT	implicit multiplication is not allowed
48	R6	"(1)(2)"	REJECT	REJECT	implicit multiplication is not allowed
49	R7	""	REJECT	REJECT	the empty string
50	R7	"5"	ACCEPT	ACCEPT	the shortest valid expression
51	RX	"3 4"	REJECT	REJECT	a space is not in Sigma
52	RX	"1a"	REJECT	REJECT	a letter is not in Sigma
53	RX	"3.5"	REJECT	REJECT	a decimal point is not in Sigma

Based on these test cases, here is the observed console output with a coverage summary at the end to show every rule of the specification being covered by at least one passing test:

```
C/C++
   #  Rule  Input                          Expected  Observed   Result  Note
  ------------------------------------------------------------------------------------------
   1   R6   "3+4"                        ACCEPT  ACCEPT  PASS     a number, an operator, a
  number
   2   R1   "(3+4)*(2-(1+5))"            ACCEPT  ACCEPT  PASS     the example: balanced nested
  parentheses
   3   R4   "-5+3"                       ACCEPT  ACCEPT  PASS     the example: unary minus at
  the start
   4   R5   "0+1"                        ACCEPT  ACCEPT  PASS     the example: a single 0 is
  allowed
   5   R3   "3++4"                       REJECT  REJECT  PASS     the example: two operators
  consecutively
   6   R2   "+3"                         REJECT  REJECT  PASS     the example: operator at the
  start
   7   R2   "3+"                         REJECT  REJECT  PASS     the example: operator at the
  end
   8   R1   "(3+4"                       REJECT  REJECT  PASS     the example: unbalanced
  parentheses
   9   R5   "007+1"                      REJECT  REJECT  PASS     the example: leading zeros
   10  R3   "3*(/2)"                     REJECT  REJECT  PASS     the example: an operator
  after ( with no operand
   11  R1   "(3+4)"                      ACCEPT  ACCEPT  PASS     one balanced group
   12  R1   "((1+2)*(3+4))"              ACCEPT  ACCEPT  PASS     two levels of nesting
   13  R1   "((1))"                      ACCEPT  ACCEPT  PASS     a group around a group
   14  R1   "3+4)"                       REJECT  REJECT  PASS     one closing parenthesis too
  many
   15  R1   "(()))"                      REJECT  REJECT  PASS     an empty group and a stray )
   16  R1   ")("                         REJECT  REJECT  PASS     a closing parenthesis with no
  opener
   17  R1   "(1+2))"                     REJECT  REJECT  PASS     the second group is never
  closed
   18  R1   "()"                         REJECT  REJECT  PASS     an empty group has no operand
   19  R2   "*2"                         REJECT  REJECT  PASS     an operator at the start
   20  R2   "/2"                         REJECT  REJECT  PASS     an operator at the start
   21  R2   "3*"                         REJECT  REJECT  PASS     an operator at the end
   22  R2   "3/"                         REJECT  REJECT  PASS     an operator at the end
   23  R2   "-"                          REJECT  REJECT  PASS     a lone minus is not an
  operand
   24  R3   "3**4"                       REJECT  REJECT  PASS     two multiplication signs
   25  R3   "3//4"                       REJECT  REJECT  PASS     two division signs
   26  R3   "3+-4"                       ACCEPT  ACCEPT  PASS     a unary minus may follow an
  operator
   27  R3   "3--4"                       ACCEPT  ACCEPT  PASS     a unary minus may follow an
  operator
```

```
 28  R3   "3+*4"                 REJECT  REJECT  PASS     the second operator is not a
minus
 29  R3   "(*2)"                 REJECT  REJECT  PASS     an operator straight after an
opener
 30  R4   "5*-3"                 ACCEPT  ACCEPT  PASS     a unary minus after an
operator
 31  R4   "3*(-2)"               ACCEPT  ACCEPT  PASS     a unary minus after an
opening parenthesis
 32  R4   "-(3+4)"               ACCEPT  ACCEPT  PASS     a unary minus on a group at
the start
 33  R4   "5/-3"                 ACCEPT  ACCEPT  PASS     a unary minus after a
division sign
 34  R4   "(-3)"                 ACCEPT  ACCEPT  PASS     a unary minus inside a group
 35  R5   "0"                    ACCEPT  ACCEPT  PASS     the single digit 0 is allowed
 36  R5   "0-5"                  ACCEPT  ACCEPT  PASS     a zero followed by an
operator
 37  R5   "01"                   REJECT  REJECT  PASS     a leading zero
 38  R5   "00"                   REJECT  REJECT  PASS     leading zeros
 39  R5   "000"                  REJECT  REJECT  PASS     leading zeros
 40  R5   "10"                   ACCEPT  ACCEPT  PASS     a zero inside a number is
fine
 41  R5   "100+20"               ACCEPT  ACCEPT  PASS     zeros inside numbers are fine
 42  R6   "42"                   ACCEPT  ACCEPT  PASS     a single number is a complete
expression
 43  R6   "10/2"                 ACCEPT  ACCEPT  PASS     a two-digit operand
 44  R6   "1234567890"           ACCEPT  ACCEPT  PASS     a long number
 45  R6   "1+2+3+4+5"            ACCEPT  ACCEPT  PASS     a chain of binary operators
 46  R6   "1/0"                  ACCEPT  ACCEPT  PASS     only the syntax is checked,
not the value
 47  R6   "2(3)"                 REJECT  REJECT  PASS     implicit multiplication is
not allowed
 48  R6   "(1)(2)"               REJECT  REJECT  PASS     implicit multiplication is
not allowed
 49  R7   ""                     REJECT  REJECT  PASS     the empty string
 50  R7   "5"                    ACCEPT  ACCEPT  PASS     the shortest valid expression
 51  RX   "3 4"                  REJECT  REJECT  PASS     a space is not in Sigma
 52  RX   "1a"                   REJECT  REJECT  PASS     a letter is not in Sigma
 53  RX   "3.5"                  REJECT  REJECT  PASS     a decimal point is not in
Sigma
-------------------------------------------------------------------------------------------
Coverage of the rules of the specification
-------------------------------------------------------------------------------
  R1  parentheses are balanced and properly nested           10 / 10 passed
  R2  no operator at the start or at the end of the expression 7 / 7 passed
  R3  no two operators in a row                               8 / 8 passed
  R4  a unary minus only at the start, after ( or after an operator 6 / 6 passed
  R5  no digit sequence with a leading zero unless it is exactly 0 9 / 9 passed
  R6  a well-formed expression: operands and operators alternate 8 / 8 passed
  R7  a boundary condition of the specification               2 / 2 passed
  RX  a character that is not in the alphabet                3 / 3 passed
```

```
--------------------------------------------------------------------------------
53 / 53 test cases passed.
```

The transition table was also cross-checked against an independent recursive-descent parser that implements the grammar of Section IV directly. Both recognisers were run on every short string (up to length 8) over three small alphabets and agreed on all of them with zero mismatches.

Bugs found while testing and how they were fixed.

Bugs found while testing and how they were fixed.

- The first version of the transition table had the stack-top symbols in the wrong order, so a) popped the bottom marker Z instead of (and the program crashed. The table was corrected, and a guard now prevents the bottom marker from ever being popped.
- The first prototype did not use a transition table, so it could not be compared with the table in Section V. It was replaced by the table-driven simulator, so the program, the table, and the diagram describe the same automaton.
- The test table of this section lists exactly the same cases, in the same order, as the test list built into the program, and the console output below was captured from the program itself.

The console output of the program for the ten examples of the problem statement, one command line per example, is:

```C/C++
    "3+4" -> ACCEPT
    "(3+4)*(2-(1+5))" -> ACCEPT
    "-5+3" -> ACCEPT
    "3++4" -> REJECT
    "+3" -> REJECT
    "3+" -> REJECT
    "(3+4" -> REJECT
    "007+1" -> REJECT
    "0+1" -> ACCEPT
    "3*(/2)" -> REJECT
```

Limitations

- **Syntax only.** 1/0 is accepted because the validator does not evaluate the expression, and operator precedence is not represented.
- **Repeated unary minus is accepted.** The rules allow a unary minus after an operator, so --5 and 3---4 are accepted.
- **Integers only.** Decimals, exponents, and variables are rejected.
- **No whitespace tolerance.** 3 + 4 is rejected; spaces must be removed first.
- **ASCII input only.** Any other character, including Unicode digits, is rejected.
- **Nesting depth is limited by memory.** The stack is unbounded in theory, but a very deeply nested input is limited by available memory in practice.
- **No reason for rejection.** The validator reports only accept or reject, not which rule was broken.

Appendix A: Division of Work

Member	Contribution

Appendix B: Source Code

C/C++

```
// Arithmetic Expression Validator
// This program checks if a string is a valid arithmetic expression,
// using the idea of a Push-Down Automaton (PDA):
//   - the machine is always in one "state"
//   - it reads the input one character at a time
//   - it uses a stack to remember open parentheses
//
// Allowed characters: 0-9  +  -  *  /  (  )
//
// Rules:
//   1. Parentheses must be balanced:          (3+4)   ok,  (3+4   bad
//   2. No operator at the start or the end:   +3      bad, 3+     bad
//   3. No two operators in a row:             3++4    bad
//      (except a minus sign:                  3+-4    ok)
//   4. A minus sign may start a number:       -5+3    ok,  3*(-2) ok
//   5. No leading zeros:                      0       ok,  007    bad
//   6. No implicit multiplication:            2(3)    bad

#include <iostream>
#include <stack>
#include <string>

using namespace std;

// ---------------------------------------------------------------------------
// The states of the machine
// ---------------------------------------------------------------------------
const int EXPECT_NUMBER   = 0;  // q0: waiting for a number, '(' or '-'
const int IN_NUMBER       = 1;  // q1: in the middle of a number (like "12")
```

```
const int EXPECT_OPERATOR = 2;  // q2: a number or ')' just ended
const int DEAD            = 3;  // qD: a rule was broken, the input is rejected

string stateName(int state)
{
    if (state == EXPECT_NUMBER)   return "q0";
    if (state == IN_NUMBER)       return "q1";
    if (state == EXPECT_OPERATOR) return "q2";
    return "qD";
}

bool isDigit(char c)
{
    return c >= '0' && c <= '9';
}

bool isOperator(char c)
{
    return c == '+' || c == '-' || c == '*' || c == '/';
}

// ------------------------------------------------------------------------
// One step of the machine: read character c while in "state",
// update the stack if needed, and return the new state.
// ------------------------------------------------------------------------
int nextState(int state, char c, stack<char>& parens)
{
    if (state == EXPECT_NUMBER) {
        if (c == '0') {
            // A single 0. No digit may follow it (no leading zeros),
            // so we go straight to "expect operator".
            return EXPECT_OPERATOR;
        }
        if (isDigit(c)) {
            return IN_NUMBER;          // a number starting with 1-9
        }
        if (c == '-') {
            return EXPECT_NUMBER;      // a minus sign in front of a number
        }
        if (c == '(') {
            parens.push('(');          // remember the open parenthesis
            return EXPECT_NUMBER;
        }
        return DEAD;                   // '+', '*', '/', ')' or a bad character
    }
```

```
    if (state == IN_NUMBER || state == EXPECT_OPERATOR) {
        if (isDigit(c)) {
            // More digits are fine inside a number like "123".
            // After a 0 or a ')' a digit is not allowed.
            if (state == IN_NUMBER) {
                return IN_NUMBER;
            }
            return DEAD;
        }
        if (isOperator(c)) {
            return EXPECT_NUMBER;      // now we need another number
        }
        if (c == ')') {
            if (parens.empty()) {
                return DEAD;           // nothing to close
            }
            parens.pop();              // close the last open parenthesis
            return EXPECT_OPERATOR;
        }
        return DEAD;                   // '(' right after a number, or a bad
character
    }

    return DEAD;                       // once dead, always dead
}

// Show the stack as text, for example "Z((" means two open parentheses.
// Z is the bottom of the stack.
string stackToString(const stack<char>& parens)
{
    return "Z" + string(parens.size(), '(');
}

// ---------------------------------------------------------------------------
// Run the machine on the whole input.
// Returns true if the expression is valid.
// If showSteps is true, every step is printed.
// ---------------------------------------------------------------------------
bool checkExpression(const string& input, bool showSteps)
{
    stack<char> parens;                // the PDA stack
    int state = EXPECT_NUMBER;         // the start state

    if (showSteps) {
```

```
        cout << "\nStep  (state, input left, stack)\n";
        cout << "--------------------------------\n";
        cout << " 0:   (" << stateName(state) << ", " << input << ", "
             << stackToString(parens) << ")\n";
    }

    for (size_t i = 0; i < input.size(); i++) {
        char c = input[i];
        state = nextState(state, c, parens);

        if (showSteps) {
            string rest = input.substr(i + 1);
            if (rest.empty()) {
                rest = "e";            // e means "nothing left" (epsilon)
            }
            cout << " " << (i + 1) << ":   (" << stateName(state) << ", " << rest
                 << ", " << stackToString(parens) << ")   read '" << c << "'\n";
        }

        if (state == DEAD) {
            return false;              // stop early, a rule was broken
        }
    }

    // At the end of the input the expression is valid only if:
    //   - the last thing read was a number or a ')'
    //   - every '(' has been closed
    bool endedWell = (state == IN_NUMBER || state == EXPECT_OPERATOR);
    return endedWell && parens.empty();
}

// ---------------------------------------------------------------------------
// Built-in tests
// ---------------------------------------------------------------------------
struct Test {
    string input;
    bool shouldAccept;
};

void runTests()
{
    Test tests[] = {
        // examples from the problem
        {"3+4", true},           {"(3+4)*(2-(1+5))", true},
        {"-5+3", true},          {"0+1", true},
```

```
    {"3++4", false},          {"+3", false},
    {"3+", false},            {"(3+4", false},
    {"007+1", false},         {"3*(/2)", false},
    // parentheses
    {"(3+4)", true},          {"((1+2)*(3+4))", true},
    {"((1))", true},          {"3+4)", false},
    {"((()))", false},        {")(", false},
    {"(1+2))", false},        {"()", false},
    // operators at the start or end
    {"*2", false},            {"/2", false},
    {"3*", false},            {"3/", false},
    {"-", false},
    // two operators in a row
    {"3**4", false},          {"3//4", false},
    {"3+-4", true},           {"3--4", true},
    {"3+*4", false},          {"(*2)", false},
    // minus sign in front of a number
    {"5*-3", true},           {"3*(-2)", true},
    {"-(3+4)", true},         {"5/-3", true},
    {"(-3)", true},
    // leading zeros
    {"0", true},              {"0-5", true},
    {"01", false},            {"00", false},
    {"000", false},           {"10", true},
    {"100+20", true},
    // general
    {"42", true},             {"10/2", true},
    {"1234567890", true},     {"1+2+3+4+5", true},
    {"1/0", true},            {"2(3)", false},
    {"(1)(2)", false},
    // edge cases and bad characters
    {"", false},              {"5", true},
    {"3 4", false},           {"1a", false},
    {"3.5", false},
};

int total = sizeof(tests) / sizeof(tests[0]);
int passed = 0;

cout << "\n";
for (int i = 0; i < total; i++) {
    bool result = checkExpression(tests[i].input, false);
    bool ok = (result == tests[i].shouldAccept);
    if (ok) {
        passed++;
```

```
        }

        cout << (ok ? "PASS" : "FAIL") << "  \"" << tests[i].input << "\"
expected "
             << (tests[i].shouldAccept ? "ACCEPT" : "REJECT") << ", got "
             << (result ? "ACCEPT" : "REJECT") << "\n";
    }

    cout << "\n" << passed << " / " << total << " tests passed.\n";
}

// ---------------------------------------------------------------------------
// Main program with a simple menu
// ---------------------------------------------------------------------------
int main(int argc, char* argv[])
{
    // Optional command line use:
    //   ./program "3+4"     checks one expression
    //   ./program --test    runs the tests
    if (argc > 1) {
        string arg = argv[1];
        if (arg == "--test") {
            runTests();
        } else {
            cout << "\"" << arg << "\" -> "
                 << (checkExpression(arg, false) ? "ACCEPT" : "REJECT") << "\n";
        }
        return 0;
    }

    cout << "=== Arithmetic Expression Validator (PDA) ===\n";

    while (true) {
        cout << "\n1. Check an expression\n";
        cout << "2. Check an expression step by step\n";
        cout << "3. Run the tests\n";
        cout << "4. Exit\n";
        cout << "Choose an option: ";

        string choice;
        if (!getline(cin, choice)) {
            break;                      // end of input
        }

        if (choice == "1" || choice == "2") {
```

```
            string expression;
            cout << "Enter an expression: ";
            getline(cin, expression);

            bool showSteps = (choice == "2");
            bool valid = checkExpression(expression, showSteps);
            cout << "\nResult: " << (valid ? "ACCEPTED" : "REJECTED") << "\n";
        } else if (choice == "3") {
            runTests();
        } else if (choice == "4") {
            cout << "Goodbye.\n";
            break;
        } else {
            cout << "Invalid choice. Please choose 1, 2, 3 or 4.\n";
        }
    }

    return 0;
}
```